Ders 15 Genişletilmiş Notu: IEEE 802 LAN Teknolojileri: Ethernet, Token Bus, Token Ring ve FDDI

Ders 15 Genişletilmiş Notu: IEEE 802 LAN Teknolojileri: Ethernet, Token Bus, Token Ring ve FDDI

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Giriş: Veri Bağlantı Katmanından Yerel Ağlara
- 2. Noktadan Noktaya ve Çok Noktalı İletişim
- 3. 802 Ailesine Genel Bakış: Kablolu, Kablosuz ve Kişisel Alan Ağları
- 4. IEEE 802 Projesi ve Veri Bağlantı Katmanının İki Alt Katmanı
- 5. ALOHA ve Slotlu ALOHA
- 6. Taşıyıcı Dinlemeli Çoklu Erişim (CSMA)
- 7. Çarpışma Algılama: CSMA/CD
- 8. IEEE 802.3 Ethernet
- 9. IEEE 802.4 Token Bus
- 10. IEEE 802.5 Token Ring
- 11. FDDI (Fiber Distributed Data Interface)
- 12. Örnek Sorular

1. Giriş: Veri Bağlantı Katmanından Yerel Ağlara

Hoca derse bir önceki haftayı hatırlatarak başladı. Geçen hafta senkron ve asenkron veri bağlantı (data link) protokolleri anlatılmış, HDLC ayrıntılı olarak incelenmişti. HDLC'nin kendisi, OSI referans modeline benzer şekilde, başka protokollerin esinlendiği bir prototip model olarak sunulmuştu. Bu hafta da hâlâ veri bağlantı katmanının içindeyiz. Fark şu: Artık yerel ağ (LAN, Local Area Network) teknolojilerine geçiyoruz.

Bu derste IEEE'nin 802 ile başlayan protokollerine "göz gezdirilecek": 802.3 Ethernet, 802.4 Token Bus, 802.5 Token Ring ve fiber optik kablolar için geliştirilmiş ayrı bir arayüz olan FDDI. Kablosuz yerel ağlar (802.11) bir sonraki haftanın konusu.

Hoca ortam erişim (media access) tekniklerinin üç sınıfta toplandığını söyledi:

- **Sabit atamalı (fixed based)** yöntemler,
- **Çekişmeli (contention based)** yöntemler,
- **Token veya rezervasyon tabanlı** yöntemler. Token Ring ve Token Bus bu gruptadır.

2. Noktadan Noktaya ve Çok Noktalı İletişim

LAN protokolleri çok noktalı (multipoint) modda çalışır. Hoca bu farkı, vizede de sorulan bir soru üzerinden açıkladı.

Noktadan noktaya (point to point) iletişimde bir ben varım, bir de benimle konuşan tek bir nokta var. Ortamda başka kimse yoksa birbirimizin adresini bilmemize gerek yoktur. Ben gönderdiğimde karşımda zaten tek bir muhatap vardır; o da cevabı bana gönderir.

Çok noktalı iletişimde ise durum farklıdır. Ağda benim dışımda 15 cihaz varsa ve ben bunlardan birine veri göndermek istiyorsam, o cihazın adresini de söylemem gerekir. LAN mimarileri bu şekilde çalışır.

> **Hocanın vurgusu:** Noktadan noktaya iletişimde adres bilgisi gerekmez. Çok noktalı iletişimde alıcının adresi verilmelidir. Vizede bununla ilgili bir soru sorulmuştu.

3. 802 Ailesine Genel Bakış: Kablolu, Kablosuz ve Kişisel Alan Ağları

Hoca ilk olarak 802 projesinin içindeki farklı alanları kısaca tanıttı. 802.1 ve 802.2 gibi kısımlar da vardır, ama ders açısından anlamlı olan kısım 802.3 Ethernet ile başlar. Sırasıyla Ethernet'in hangi yıllarda, hangi amaçla geliştirildiği, ardından token kullanan sürümler anlatılacak.

Token Bus, bus topolojisi için geliştirilmiştir. Hoca burada topolojileri hatırlattı: Bus tek bir hattır; kademeli bus (cascade bus) yapısına ise ağaç (tree) diyorduk. Token Bus her ikisinde de kullanılabilir. Token Ring ise halka (ring) topolojisi için geliştirilmiş sürümdür.

3.1 Kablosuz Yerel Ağ ve Kişisel Alan Ağları

Kablosuz yerel ağ (wireless LAN) bugün çok yaygındır. Hoca, derse bağlandığı makinenin de büyük ihtimalle 802.11'in n, g gibi sürümlerinden biriyle bağlı olduğunu söyledi. Ama kablosuz dünya yalnız Wi-Fi'dan ibaret değildir.

Bluetooth da bir kablosuz teknolojidir. Evdeki bir Wi-Fi ağına on makine bağlanıyor olabilir, ama günlük hayatta belki daha çok Bluetooth ile bağlanıyoruz. Hocanın örnekleri:

- Akıllı saatimiz telefona Bluetooth ile bağlanır.
- Kablosuz kulaklığımız telefona Bluetooth ile bağlanır.
- Otomobile oturup telefondan açtığımız müziği arabada dinlerken yine Bluetooth kullanılır.

Hoca ZigBee'yi Bluetooth'un özelleşmiş bir sürümü olarak tanıttı ve ikisinin de 802.15 projesinin kapsamında olduğunu söyledi. Bunlar kişisel alan ağlarında (PAN, Personal Area Network) kullanılır.

> **Ders sonrası not:** Hoca ZigBee'nin Bluetooth'un özelleşmiş bir sürümü olduğunu, temelde Bluetooth tabanlı düşünülebileceğini söyledi. İkisi ayrı teknolojilerdir. ZigBee büyük olasılıkla IEEE 802.15.4 tabanlıdır. Bluetooth ise başlangıçta büyük olasılıkla 802.15.1 olarak standartlaştırılmıştı. Ortak noktaları, ikisinin de 802.15 kişisel alan ağı ailesinde yer almasıdır.

3.2 WiMAX

Hoca son olarak WiMAX'tan söz etti. 2007–2010 yıllarında WiMAX büyük bir beklenti yaratmıştı. "WiMAX gelecek, internet sorunu çözülecek" deniyordu. Bugün Starlink'in uydudan internet verme projesi nasıl konuşuluyorsa, o yıllarda WiMAX öyle konuşuluyordu.

WiMAX'ı bir baz istasyonu gibi düşünebiliriz. Anteninin gücüne göre 5 km, 10 km, belki 50 km yarıçaplı bir alana kablosuz ağ yayan devasa antenlerdir. Hocanın benzetmesiyle, kampüsün orta bahçesine dikilmiş kocaman bir direk her yere ağ yayar; kampüsün her yerinden, hatta dışarıdan bağlanabilirsiniz.

Proje haline getirilmiş, bazı prototipleri uygulanmıştır. Ama mobil ağların yaygınlaşması ve hızlarının artmasıyla (4.5G, bazı ülkelerde 5G) WiMAX hocaya göre gerçekçi biçimde hayata geçmemiş, bugün kullandığımız teknolojilerden biri değildir.

> **Ders sonrası not:** Hoca WiMAX'ı geniş bir alana "Wi-Fi ağı yayan" antenler olarak anlattı. WiMAX, Wi-Fi (802.11) değil, IEEE 802.16 standardıdır. Hoca hayata geçirilmediğini söyledi; büyük olasılıkla birçok ülkede ticari olarak kuruldu, ama zamanla yerini LTE gibi mobil teknolojilere bıraktı.

4. IEEE 802 Projesi ve Veri Bağlantı Katmanının İki Alt Katmanı

Bir LAN içinde birçok farklı protokol kullanılabilir. Farklı makinelerin, istasyonların üzerinde farklı ağ arayüz denetleyicileri (NIC, Network Interface Controller), yani farklı ağ kartları olabilir. Bunların da kendi protokolleri olabilir. Bütün bunların birbiriyle uyumlu konuşabilmesi için IEEE 802 projesi geliştirilmiştir.

4.1 Katmanlar

IEEE 802 projesinde veri bağlantı katmanı iki ayrı alt katmandan oluşur: ortam erişim denetimi (MAC, Media Access Control) ve mantıksal bağlantı denetimi (LLC, Logical Link Control). Hoca uygulama, oturum gibi üst katmanları geçerek ağ katmanından aşağısını şöyle özetledi:

- **802.1:** Ağlar arası çalışma (internetworking) protokolü. Hoca bunu ağ katmanının içinde gösterdi.
- **802.2:** Mantıksal bağlantı denetimi (LLC) protokolü. Veri bağlantı katmanının içindedir.
- **MAC protokolleri:** Yine veri bağlantı katmanının içindedir. CSMA/CD, Token Bus ve Token Ring burada yer alır.

Bu MAC protokollerini gerçekleştirebilmek için fiziksel katmanda da uygun bir altyapı gerekir; yani protokolü destekleyecek bir kablo yapısı. Hocanın örneği: Bir protokol koaksiyel kabloda 50 ohm'luk bir kablo istiyorsa altyapıyı o kabloyla kurmalısınız ki MAC protokolü de desteklenmiş olsun.

4.2 LLC Hizmet Türleri

Hoca LLC'nin üç hizmet türünü anlattı:

- **Onaysız bağlantısız hizmet (unacknowledged connectionless service):** Karşı taraftan onay (ACK) alınıp alınmaması önemli değildir. Hocanın ifadesiyle: "Veriyi iletiyorum, ulaşmışım ulaşmamışım çok umursamıyorum."
- **Bağlantı yönelimli hizmet (connection mode service):** Karşı tarafla bağlantıyı kurduğumu biliyorum. Bir mesaj gönderdiğimde, mesajın onun eline ne zaman geçeceğini biliyorum.
- **Onaylı bağlantısız hizmet (acknowledged connectionless service):** Mesajı gönderdikten sonra karşı taraf bana mesajın geçip geçmediğini veya hatalı geçtiğini bildirir. Zaman aşımı (timeout) durumunda da bir tepki verir.

4.3 LLC PDU Yapısı

Gönderdiğimiz çerçevenin (frame) içinde başlangıç ve bitiş sınırlayıcıları (start/end delimiter) gibi denetim alanları olduğunu hatırlayalım. Çerçevenin amacı karşıya veri göndermektir. Bu veriye protokol veri birimi (PDU, Protocol Data Unit) denir. LLC PDU'su şu alanlardan oluşur:

- Hedef hizmet erişim noktası (DSAP, Destination Service Access Point),
- Kaynak hizmet erişim noktası (SSAP, Source Service Access Point),
- Denetim alanı (control field),
- Bilgi alanı (information).

Hoca denetim alanının ilerleyen kısımlarda da karşımıza çıkacağını, örneğin yük bindirme (piggybacking) için bu alanların kullanıldığını hatırlattı.

5. ALOHA ve Slotlu ALOHA

5.1 Tarihçe: Adalar Arası İletişim

Hoca 802.3'ün hikâyesine 1970'lerin başından başladı. Xerox şirketi ağ kapasitesi açısından neler yapılabileceğini konuşuyordu. Ardından Hawaii'ye geçti. Hawaii, birbirine yakın birçok küçük adadan oluşur ve adaların birbiriyle iletişim kurması gerekir.

Geleneksel yöntem neydi? Ateş ya da ışık yakarak Mors alfabesiyle haberleşmeyecekseniz ve elinizde ayrıntılı bir metin varsa, tekneye binip metni karşıya götürürsünüz. Karşıdaki okur,

cevabını yazar, siz de cevabı tekneyle geri getirirsiniz. Soru şuydu: Bu iletişimi karşı tarafa fiziksel olarak geçmeden, daha etkin biçimde yapabilir miyiz?

Bunun için kurulan ilk ağ ALOHA'dır. Hoca ALOHA'nın 1973'te Bob Metcalfe öncülüğünde Hawaii adalarında kurulduğunu söyledi. Ağ, radyo frekansları ve radyo antenleri üzerinden kablosuz çalışıyordu.

> **Ders sonrası not:** ALOHA'nın kurucusu ve tarihi konusunda hoca iki ayrı hikâyeyi karıştırdı. ALOHAnet büyük olasılıkla Hawaii Üniversitesi'nde Norman Abramson ve ekibi tarafından geliştirildi ve 1971'de çalışmaya başladı. Bob Metcalfe ise büyük olasılıkla 1973'te Xerox PARC'ta, ALOHA'dan esinlenerek Ethernet'i geliştirdi. Hocanın andığı Xerox bağlantısı Ethernet'e aittir.

5.2 ALOHA Nasıl Çalışır?

Hoca on adaya anten kurulduğunu varsaydı. Göndermek istediğim veriyi kendi antenimden yayınlarım. Diğer herkes bu yayını duyar. Mesaj kendisine gönderildiyse onu dinler ve alır, gerekli işlemleri yapar; yolda bir şey olursa alamaz. Cevap verecekse bana aynı şekilde cevap gönderir. Böylece ağdaki herkes ağda olan biteni dinleyebilir.

Sorun şuradadır: On ada var. Ben 3 numaralı adayım ve 7 numaralı adaya veri gönderiyorum. Aynı anda 8 numaralı ada da 2 numaralı adaya gönderirse bu bir problemdir. Sinyaller havada aynı ya da benzer frekanslarda ilerlediği için havada bir karmaşa oluşur. Hocanın benzetmesi: Ben konuşurken yanımda biri daha konuşursa ikimizin dediği de anlaşılmaz.

Buna verinin çarpışması (collision) denir: Verinin havada veya kablonun içinde çarpışıp hiçbir taraf için anlam taşımayan bir hale gelmesi. Hoca bunun paylaşılan ortamdaki en ciddi sorunlardan biri olduğunu söyledi.

Çarpışmalar yüzünden ALOHA'da hattın kullanım oranı (utilization) yaklaşık %18'dir. Hiç çarpışma olmadan hat %100 kullanılabilecekken, aynı anda gönderilen verilerin çarpışması yüzünden kullanım oranı aşağı yukarı beşte bire düşer. Sorunun kaynağı, hocanın deyişiyle, "kafasına esen herkesin" istediği an göndermesidir.

5.3 Slotlu ALOHA

Bu sorunu azaltmak için slotlu ALOHA (slotted ALOHA) geliştirildi. Adı üzerinde, zaman slotlara bölünür. Hoca bunu şu örnekle anlattı: Her saniye 10 parçaya bölünmüş olsun; istasyon başına 100 milisaniye düşer. Örneğin 7. istasyona şöyle denir: "Saniyenin 600 ile 700. milisaniyeleri arasında veri gönder. Göndermek istemiyorsan sus. Daha fazla verin varsa ve süre yetmiyorsa bir sonraki saniyenin aynı slotunu bekle."

Bu sayede kullanım oranı iki katından fazla artmış ve yaklaşık %37'ye çıkmıştır.

Hoca burada öğrencilerin aklına gelebilecek bir soruyu kendisi sordu: "Herkes kendi zamanında gönderiyorsa çarpışma olmaz, kullanım oranı %100 olmalı değil mi?" Hocanın cevabı zaman senkronizasyonuydu. Her makine saatini milisaniye cinsinden ayarlar, ama çevresindeki elektromanyetik alan gibi etkiler yüzünden saatlerde sapmalar olur. Bir istasyon farkında olmadan başkasının slotuna taşar. Öbür istasyon da o slotu kendisininki sanarak gönderir ve veriler çarpışır. Senkronizasyon için önlemler alınsa da, senkronizasyonun bozulduğu durumlar hesaba katıldığında kullanım oranı %37'de kalmıştır.

> **Ders sonrası not:** Hoca slotlu ALOHA'yı her istasyona sabit bir slot verilen bir düzen gibi anlattı ve %37 sınırını saat kaymasına bağladı. Her istasyona sabit slot vermek zaman bölmeli çoğullamadır (TDMA). Slotlu ALOHA'da herhangi bir istasyon herhangi bir slotun başında göndermeye başlayabilir; aynı slotu seçen istasyonlar yine çarpışır. Verimin iki katına çıkmasının nedeni, çarpışmaya açık sürenin (vulnerable time) iki çerçeve süresinden bir çerçeve süresine inmesidir. %36,8 sınırı bu olasılık hesabından gelir, saat kaymasından değil. Sınır değerleri Python ile doğrulandı: $1/(2e) \approx 0{,}184$ ve $1/e \approx 0{,}368$.
>
> **Ek bilgi (derste anlatılmadı):** ALOHA analizinde G, bir çerçeve süresinde ağa sunulan ortalama çerçeve sayısıdır (yeniden gönderimler dahil). Saf ALOHA'da başarılı iletim oranı $S = Ge^{-2G}$'dir ve en büyük değeri $G = 0{,}5$'te $S_{max} = \frac{1}{2e} \approx 0{,}184$ olur.

Slotlu ALOHA'da $S = Ge^{-G}$'dir ve en büyük değeri $G = 1$'de $S_{max} = \frac{1}{e} \approx 0{,}368$ olur. Derste geçen %18 ve %37 değerleri bu iki sınırdır.

6. Taşıyıcı Dinlemeli Çoklu Erişim (CSMA)

6.1 Temel Fikir

ALOHA'yı slotlu ALOHA ile iyileştirdik. Slotlu ALOHA'yı da iyileştirebilir miyiz? Hoca şu gözlemden yola çıktı: İstasyonun bir gönderici anteni, bir de alıcı anteni var. Alıcı antenle veri alabiliyorum. O halde aynı antenle hattı da dinleyebilir miyim? Yani hatta şu an bir gönderim olup olmadığını dinleyip, hat müsait olduğunda gönderebilir miyim? Taşıyıcı dinlemeli çoklu erişim (CSMA, Carrier Sense Multiple Access) bu fikri gerçekleştirir. Hoca bunun üç farklı sürümünü anlattı.

6.2 Birinci Sürüm: Hat Boşalınca Hemen Gönder

Hoca ilk sürümü "ısrarsız CSMA (non-persistent CSMA)" diye adlandırdı. Bu sürümde istasyon hattı dinler. Hat meşgulse göndermez, hattın müsait olmasını bekler. Müsait olduğu anda göndermeye başlar.

Sorun şudur: Benim dışımda gönderecek verisi olan başka istasyonlar da hattı dinlemektedir. Hocanın örneğinde her biri hattı 10 milisaniyede bir kontrol ediyor. On makineden ikisi kendi arasında konuşurken kalan sekiz makineden üçünün bir şey göndermek istediğini düşünelim. Üçü de dinler: meşgul. Yine dinler: meşgul. Sonunda üçü birden hattı müsait bulur ve üçü birden göndermeye başlar. Yine çarpışma olur. Hoca bunun gerçekçi bir problem olduğunu vurguladı.

6.3 İkinci Sürüm: Hat Boşaldıktan Sonra Bekle ve Yeniden Kontrol Et

Çözüm önerisi: Hat müsait olduktan sonra her makine belirli bir süre beklesin. Örneğin 3 numaralı makine 30 milisaniye, 7 numaralı makine 70 milisaniye beklesin. Herkese önceden tanımlı farklı süreler verilebilir; hepsi aynı süre de bekleyebilir.

Hoca burada eksik anlattığını düşündüğü bir noktayı düzeltti: Bekledikten sonra doğrudan gönderilmez, hat yeniden kontrol edilir. 10 milisaniye bekleyen istasyon süresi dolunca hattı tekrar kontrol eder, müsaitse gönderir. 20 milisaniye bekleyen istasyon süresi dolup hattı kontrol ettiğinde, 10 milisaniye bekleyen göndermeye başladığı için hattı meşgul bulur. Bu yöntem çarpışma oranını azaltır ve daha kararlı bir sonuç verir. Ama bazı istasyonların bekleme süreleri yine çakışabilir. Kullanım oranı önceki sürüme göre artar; daha da artırılabilir mi?

6.4 Üçüncü Sürüm: Rastgele Bekleme

Hoca üçüncü sürümü "p-ısrarcı CSMA (p-persistent CSMA)" diye adlandırdı. İstasyon hattı kontrol eder. Hat müsaitse rastgele bir süre bekler; örneğin 0 ile 1000 milisaniye arasında rastgele bir değer üretilir. Hoca bu değerin karmaşık bir formülü olduğunu, göndermek istediğimiz yerin uzaklığı ve altyapı gibi birçok parametreye bağlı olabileceğini, ama ayrıntının önemli olmadığını söyledi. Önemli olan bir rastgeleleştirme fonksiyonunun bulunmasıdır. Üretilen süre kadar beklendikten sonra hat tekrar kontrol edilir ve müsaitse gönderilir.

Hoca şunu da ekledi: Kullanım oranı %100 olmadığı sürece her zaman çarpışma vardır.

> **Ders sonrası not:** Hocanın CSMA türlerine verdiği adlar standart adlandırmayla uyuşmuyor. "Meşgulse sürekli dinle, boşalınca hemen gönder" tanımı 1-ısrarcı CSMA'ya (1-persistent) aittir. Israrsız CSMA'da (non-persistent) hat meşgulse sürekli dinlenmez; rastgele bir süre beklenip hat yeniden dinlenir. p-ısrarcı CSMA slotlu kanallarda kullanılır: Hat boşsa istasyon p olasılıkla hemen gönderir, $1 - p$ olasılıkla bir slot bekleyip yeniden dener. Hocanın anlattığı "boşsa rastgele bir süre bekle" tanımı bununla tam örtüşmez.

7. Çarpışma Algılama: CSMA/CD

7.1 Çarpışmayı Algılayabilir miyim?

Çarpışma olduğunda ne yapacağız? Ondan da önce: Çarpışmayı algılayabilir miyim? Hoca sorunu şöyle anlattı: Ben veri gönderiyorum, başka bir istasyon da gönderiyor; çarpışma oluyor ama iki taraf da göndermeye devam ediyor. Gönderici antenleri çalıştığı için gönderim sürer. Diğer istasyonlar da hattın meşgul olduğunu düşünerek bekler. Yani çarpışmış, artık anlamsız bir veri için hat boşuna işgal edilir.

Çarpışma algılamalı CSMA'da (CSMA/CD, Collision Detection) hoca çarpışmayı algılayan ayrı bir birim, ayrı bir anten olduğunu söyledi. Çarpışma algılandıktan sonra bir karıştırma sinyali (jamming signal) gönderilir.

7.2 Jam Sinyali Benzetmesi

Hoca jam sinyalini önce gerçek karıştırıcılarla (jammer) örnekledi: Önemli kişiler bir yere geldiğinde, uzaktan kumandalı düzenekleri etkisiz bırakmak için karıştırıcılar kullanılır. Bu karıştırıcıların yakınında telefon çekmez, Bluetooth çalışmaz, saatle bile bağlantı kopar. Bütün kablosuz iletişim kesilir.

İkinci benzetme bir sınıftı. Öğrenciler sınıfta kendi aralarında konuşuyor, bir uğultu var. Yakın konuşanlar belki birbirini anlıyor, ama uzaktakiler ya da dışarıdan bakan biri hiçbirini anlamıyor. Dışarıdan biri herkesten yüksek sesle bağırıyor ve herkes susuyor. Hoca jam sinyalinin hattı bu şekilde "temizlediğini" söyledi.

7.3 Akış Diyagramı

Hoca slayttaki akış diyagramını birlikte okudu:

1. Bilgisayar veri göndermeye hazırsa hattı dinler.
2. Hat kullanımdaysa tekrar dinler; hat boşalana kadar dinlemeyi sürdürür.
3. Hocanın anlatımına göre hat müsaitse önce hatta küçük bir sinyal gönderilir. Bu sinyal, hatta başka bir veri varsa onunla çarpışarak çarpışmayı ortaya çıkaracak bir sinyaldir. Ardından hat tekrar dinlenir.
4. Sorun yoksa veri gönderimine başlanır.
5. Çarpışma olmuşsa bir jam sinyali gönderilir.
6. İkili üstel geri çekilme (binary exponential backoff) algoritmasının ürettiği süre kadar beklenir.
7. İşlem başa döner.

Ders sonrası not: Hoca çarpışmayı ayrı bir antenin algıladığını, istasyonun göndermeden önce hatta küçük bir sinyal yollayıp yokladığını ve jam sinyalinin hattı "temizlediğini" anlattı. Kablolu Ethernet'te gönderici, verisini gönderirken aynı alıcı-verici (transceiver) ile hattaki sinyali izleyerek çarpışmayı algılar; göndermeden önce bir yoklama sinyali yollanmaz. Jam sinyali yalnız çarpışma algılandıktan sonra gönderilir. Amacı, çarpışmaya karışan bütün istasyonların çarpışmayı fark etmesini sağlamaktır. Hoca bekleme süresinin "milisaniye" cinsinden olduğunu söyledi; büyük olasılıkla bekleme, slot süresinin katları olarak hesaplanır ve 10 Mbps Ethernet'te bir slot $512/(10 \times 10^6) = 51{,}2$ mikrosaniyedir (Python ile doğrulandı). Kablosuz ağlarda çarpışma bu şekilde algılanamadığı için büyük olasılıkla CSMA/CA kullanılır.

Ek bilgi (derste anlatılmadı): İkili üstel geri çekilmede istasyon n. çarpışmadan sonra 0 ile $2^{\min(n,10)} - 1$ arasında rastgele bir k tamsayısı seçer ve k slot süresi bekler. Böylece her yeni çarpışmada seçim aralığı iki katına çıkar; 10. çarpışmadan sonra aralık 0–1023'te sabit kalır. Standartta büyük olasılıkla 16 başarısız denemeden sonra çerçeve bırakılır ve üst katmana hata bildirilir. Aralığın büyümesi, aynı anda göndermeye çalışan istasyonların yine aynı süreyi seçme olasılığını azaltır.

8. IEEE 802.3 Ethernet

ALOHA'dan başlayıp taşıyıcı dinleyen ve çarpışma algılayan yöntemlerle birlikte IEEE 802.3, yani Ethernet, hayatımıza girmiş oldu.

8.1 Çerçeve Yapısı

Hoca slayttaki Ethernet çerçevesini şöyle anlattı:

Alan	Boyut	Derste söylenen
Önsöz (preamble)	7 bayt	Senkronizasyon için. Aynı bayttan 7 tane gönderilir.
Çerçeve başlangıç sınırlayıcı (SFD, Start Frame Delimiter)	1 bayt	Çerçevenin nereden başladığını gösterir.
Adres alanları	Kaynak ve hedef	Hoca "başlangıç adresi, bitiş adresi" dedi.
Uzunluk	2 bayt	Verinin ne kadar uzun olduğunu söyler.
Veri	46–1500 bayt	
CRC	4 bayt	Veride ya da başka alanlarda hata olmuşsa denetim için.

Hoca önsözü "ben geliyorum" demek gibi düşünmemizi istedi: Alıcı, gelen bu düzenli bit dizisiyle saatini göndericiye uydurur.

> **Ders sonrası not:** Hoca adres alanlarını "başlangıç adresi, bitiş adresi" diye ve boyutlarını "hattınızda ne kadar bilgisayar varsa" diye anlattı. Bu alanlar hedef adres ve kaynak adrestir ve her biri sabit 6 baytlık MAC adresidir; ağdaki bilgisayar sayısına göre değişmez.
>
> **Ek bilgi (derste anlatılmadı):** Önsözün her baytı `10101010`, SFD ise `10101011` desenindedir; SFD'nin sonundaki 11 asıl çerçevenin başladığını bildirir. Hedef adres 6 bayt, kaynak adres 6 bayt, uzunluk/tip alanı 2 bayt ve CRC 4 bayttır; bu başlık ve kuyruk toplamı 18 bayt eder. Veri alanının en az 46 bayt olması sayesinde en kısa çerçeve $46 + 18 = 64$ bayt, yani 512 bit olur (Python ile doğrulandı). Bu en kısa uzunluk, gönderici çerçevenin son bitini göndermeden önce en uzak noktadaki çarpışmayı fark edebilsin diye seçilmiştir; az önce geçen 512 bitlik slot süresi de buradan gelir.

8.2 Paylaşımlı ve Anahtarlamalı Ethernet

Ethernet'in paylaşımlı (shared) ve anahtarlamalı (switched) sürümleri vardır. Hoca anahtarlamalı yapıların, anahtarlama konusu işlenirken ayrıntılı anlatılacağını söyledi.

8.3 Sürümler ve Adlandırma

802 projelerinin hepsinde olduğu gibi Ethernet'in de çok sayıda çeşidi vardır. Projede yeni bir geliştirme yapıldıkça standart adının sonundaki harf değiştirilir. Harfler Z'ye kadar gelip tükenince iki harfli adlar kullanılmaya başlanır.

- **802.3u:** Hoca bu sürümle hızın 10 Mbps'ten 100 Mbps'e çıktığını söyledi. Buna Hızlı Ethernet (Fast Ethernet) denir. Bu sürümde otomatik anlaşma (auto-negotiation) da vardır. Hoca bunu, bağlantının kurulması ve sonlandırılması için arka planda yapılan işler olarak anlattı: ACK'lar gidip gelir, iki taraf "sana veri göndereceğim zaman bu, benimki bu, seninki de buysa süper" diye konuşur.
- **802.3z ve 802.3ab:** Gigabit Ethernet. Hoca bunların CAT5'ten CAT8'e kadar kablolarla gerçekleştirildiğini ve 1 Gbps'e kadar çıkabildiğini söyledi.
- **Sonraki sürümler:** 10 Gbps'e ve 100 Gbps'e kadar çıkan sürümler vardır.

Hoca slayttaki listeye bakarak, yukarıdan aşağıya inildikçe standartların zaman içinde günümüze yaklaştığını ve Ethernet'in hızının buna göre evrimleştiğini söyledi.

> **Ders sonrası not:** Otomatik anlaşma (auto-negotiation), iki ucun ortak hız ve çift yönlülük (duplex) modu üzerinde anlaşmasıdır; ACK alışverişiyle zaman ayarı yapılması değildir. Gigabit kablolama için hoca CAT5'ten CAT8'e dedi; büyük olasılıkla

802.3ab (1000BASE-T) için en az Cat5e önerilir, 802.3z ise fiber ve kısa bakır bağlantılar içindir. Cat6A ve Cat8 gibi kablolar daha yüksek hızlar içindir.

8.4 Metro Ethernet

Hoca Metro Ethernet kavramını da tanıttı. Ethernet, LAN, yani küçük bir yerel ağ için gelişti. Metro Ethernet ise Ethernet'in bunun ötesinde kullanılmasının amaçlanmasıdır. "Metro" kelimesini metropol gibi düşünebiliriz. Hoca bunu bir şehir, bir kampüs veya bir şirketin iç ağı (intranet) gibi ağlarda Ethernet'i kullanmak olarak anlattı.

Hocanın örneği: ABC şirketinin 300 dönüm üzerine kurulu bir fabrikası var ve fabrikanın içinde yalnız iç ağ olarak kullanılan bir ağ bulunuyor. Bu ağı Metro Ethernet gibi kullanabilir miyiz? Hocaya göre Metro Ethernet bu kavramdır. Hoca kavramın prototiplerinin denenmiş olabileceğini, ama başka çözümler ihtiyacı karşıladığı için hayata geçirilmediğini söyledi ve bazı yerlerde karşımıza çıkabileceği için aktardığını belirtti.

> **Ders sonrası not:** Hoca Metro Ethernet'in hayata geçirilmemiş bir kavram olduğunu söyleyip şirket içi ağ örneği verdi. Metro Ethernet büyük olasılıkla servis sağlayıcıların şehir ölçeğinde (MAN, Metropolitan Area Network) sunduğu, bugün yaygın olarak kullanılan bir bağlantı hizmetidir. Bir şirketin kendi fabrika veya kampüs ağı bunun örneği değildir.

8.5 Ethernet Üzerinden Güç (PoE)

Hoca çok sık kullanılan bir kavram olarak Ethernet üzerinden gücü (PoE, Power over Ethernet) anlattı: Ethernet hattı üzerinden hem veri hem güç verilir.

Örnek olarak servis sağlayıcıdan alınan PoE destekli bir modemi verdi: Modemi yalnız evdeki veri girişine Ethernet kablosuyla takarsınız, prize takmazsınız, yine de çalışır; elektriğini Ethernet hattından alır. Hoca bunu sinyal bilgisine bağladı: Elektrik de bir sinyaldir. Sinyaller birbirinin üzerine bindirilerek (taşıyıcı sinyallerde olduğu gibi) gönderilebilir ve karşı tarafta Fourier teknikleri gibi yöntemlerle ayrılabilir. Böylece aynı hattan hem veri hem gerilim gönderilebilir. Bu iş Ethernet hattı üzerinden yapılırsa adı PoE olur.

Kullanım örnekleri:

- Sağa sola dönen güvenlik kameraları. Arkasına yalnız CAT5 veya CAT6 Ethernet kablosu takılır, elektriğini oradan alır.
- Robotikte kullanılan bazı sensörler. Yalnız Ethernet kablosu vardır, elektriği de onun üzerinden alırlar.
- İlkel bir örnek olarak eski sabit telefonlar. Elektrik olmadan hiçbir şey çalışmaz; sabit telefon da telefon hattından gelen enerjiyle çalışıyordu, ama daha düşük gerilimlerde.

Hoca PoE özelliğini ürün seçerken bir tercih sebebi olarak gördüğünü, çünkü evde ayrıca bir prizi işgal etmediğini söyledi. Ama cihazın enerjiyi Ethernet'ten alabilmesi için altyapının da PoE'yi desteklemesi gerekir; donanım altyapısı uygun değilse kullanılamaz. Hoca bu tür modemlerin 7-8 yıl öncesinde bile bulunduğunu, bu kavramla ileride daha sık karşılaşacağımızı söyledi.

> **Ders sonrası not:** PoE, verinin ve elektriğin Fourier teknikleriyle ayrıştırılmasıyla çalışmaz. Kablodaki tel çiftleri üzerinden doğru akım (DC) verilir; bu, veri sinyalini taşıyan transformatörlerin orta uçlarından ya da boştaki çiftlerden yapılır. Hoca "doğrudan yüksek voltaj" dedi; PoE'nin nominal gerilimi büyük olasılıkla yaklaşık 48 V'tur. Güç sınırları büyük olasılıkla 802.3af'de yaklaşık 15,4 W, 802.3at'de 30 W, 802.3bt'de 90 W civarındadır.

9. IEEE 802.4 Token Bus

9.1 Neden Token Bus?

Aradan sonra hoca 802.4'e geçti. 802.3'te ne yapıyorduk? Belli bir süre bekliyorum; bu süre rastgele olabilir, sabit olabilir, ya da hattın boşalmasını bekliyorum. Sürekli bir bekleme vardır.

Bu durum kullanım oranının belli bir noktanın üstüne çıkmasını engeller ve çarpışma ihtimali hâlâ vardır.

Bu yüzden 1980'lerin başında, General Motors tarafından Token Bus teknolojisi öne sürüldü. Hoca ALOHA ile başlayan sürecin 1972-73'lerde başladığını, Token Bus'ın ise 1980'lerin başına ait olduğunu hatırlattı.

9.2 Topoloji ve Sıralama

Token Bus, bus ve ağaç topolojilerinde kullanılabilir. Bus topolojisinde bir hat vardır, bilgisayarlar bu hatta bağlıdır.

Hocanın anlatımına göre her istasyon kendi sağındaki ve solundaki istasyonu tanır. Veri aktarılacaksa örneğin sağdakinden alıp soldakine, ya da soldakinden alıp sağdakine verir. Aktarım yapılacaksa önce en yüksek numaraya sahip istasyon başlar. En yüksek numaralı istasyon veri göndermeyecekse sıra bir düşüğüne, sonra onun bir düşüğüne geçer.

> **Ders sonrası not:** Token Bus'ta sıra, istasyonların kablo üzerindeki fiziksel komşuluğuna göre değil, adreslere göre kurulan mantıksal halkaya göredir. Her istasyon mantıksal halkadaki bir önceki ve bir sonraki istasyonun adresini bilir. Hocanın söylediği "en yüksek numaradan bir düşüğe" sıralaması bu mantıksal halkanın büyük olasılıkla azalan adres sırasıyla dolaşılmasına karşılık gelir.

9.3 Token Kavramı

> **Hocanın vurgusu:** Token kavramı önemlidir. Bugün hem Token Bus hem Token Ring görülecek; token ile çalışan başka mimariler de vardır. Token'ın nasıl çalıştığı çok önemli, dikkatle dinleyin.

Token bir anahtardır; Türkçesi jetondur. Aslında token da bir çerçevedir. Veriyi nasıl bir çerçevenin içinde gönderiyorsam token da bir çerçevedir.

Hat müsaitken hatta bir token dolaşır. Sağımdakinden token'ı alırım, soluma veririm. O da soluna verir, o da soluna verir. Hat bir bus olduğu için ters yönden yeniden bana gelebilir; ben de sağıma veririm. Böylece hatta sürekli özel bir jeton dolaşır.

Token'ı alan istasyon veri göndermek istiyorsa token'ı hattan alır ve kendi verisini hatta koyar. Bu durumda hatta artık token yoktur ve yalnız token'a sahip olan istasyon veri iletir. Doğal olarak çarpışma olma imkânı yoktur, çünkü token bendedir ve yalnız ben gönderiyorum. Veri, bus üzerindeki hedef istasyona gider. İşim bittiğinde token'ı yeniden hatta koyarım; göndermek isteyen başka istasyon varsa o token'ı alıp kendi verisini hatta koyar.

Bus topolojisinde sonradan bilgisayar eklenebilir ya da çıkarılabilir.

9.4 MAC Karmaşıklığı ve Fiziksel Katman

802.4'ün ortam erişim denetimi kısmı oldukça karmaşıktır: 10 farklı zamanlayıcı ve zamanla ilgili 24 farklı durum vardır. Bunlar senkronizasyonun sağlanmasıyla ilgilidir. Hoca ayrıntılara girmedi; bu sayıları, arka planda senkronizasyonun daha performanslı yapılabilmesi için karmaşık bir yapı bulunduğunu göstermek için verdi.

Token Bus genellikle 75 ohm'luk koaksiyel kablolar üzerinde gerçekleştirilir. Üç farklı modülasyon tekniği vardır. Hoca modülasyonu hatırlattı: Karşıya 1 ve 0'ı göndermek için fazı mı, frekansı mı, genliği mi kullanacağız? İhtiyaca ve donanım kapasitesine göre şu üç teknikten biri kullanılır:

- Faz sürekli frekans kaydırmalı anahtarlama (phase continuous FSK),
- Faz uyumlu frekans kaydırmalı anahtarlama (phase coherent FSK),
- Çok seviyeli ikili-ikili genlik modülasyonlu faz kaydırmalı anahtarlama (multilevel duobinary AM/PSK).

Hoca ayrıntıya girmeden ikisinin frekans tabanlı, birinin genlik tabanlı olduğunu bilmemizi istedi. Token Bus 10 Mbps hızlara ulaşabilir.

9.5 Çerçeve Yapısı

Token Bus çerçevesinde senkronizasyon için önsözden sonra çerçevenin nereden başladığını gösteren başlangıç sınırlayıcı (starting delimiter), bitiş sınırlayıcı (ending delimiter), denetim alanı, adres alanları ve veri alanı vardır. Hoca slaytta yanına "bayt" yazılmamış olsa da veri alanı için verilen 8182 değerinin bayt olduğunu belirtti. Hata denetimi için 4 baytlık bir CRC alanı vardır.

802.3'e göre çerçeve boyutu neredeyse 5 kat büyüktür. Hesap: $8182/1500 \approx 5{,}45$ (ara adım eklendi; Python ile doğrulandı).

9.6 Öncelik Mekanizması

Token Bus'ta bir öncelik (priority) mekanizması vardır. Ağdaki her istasyonun bir öncelik seviyesi vardır: 0, 2, 4, 6. Hoca bunu daha önce görülen birincil-ikincil (primary-secondary) sistemle karşılaştırdı: Orada iki seviyeli bir yapı vardı, burada dört seviyeye çıkarılmış. En yüksek numara, yani 6, en yüksek önceliktir; 4 daha düşüktür; 0 hiç öncelik olmadığı anlamına gelir.

Birden fazla istasyon veri göndermek istiyorsa önce en yüksek önceliğe sahip olan gönderir. En yüksek seviyede birden fazla istasyon varsa (örneğin 6. seviyede 5 makine), bunlardan önce gönderme talebinde bulunan gönderir. Hoca buna rezervasyon (reservation) diyeceğimizi ve konuyu Token Ring'de daha ayrıntılı anlatacağını söyledi.

Öncelik neden gerekir? Bazı istasyonların gönderdiği verilerin karşıya ulaşması diğerlerinden daha önemli olabilir. Hocanın örneği: Biri WhatsApp'ta metin yazıyor ya da bir müzik indiriyor. Bir başkası ise Netflix'te canlı film izliyor ya da internet üzerinden TV aboneliğiyle kanal izliyor. Para verip satın aldığınız bir hizmetin donduğunu, duraksadığını düşünün. Bir ay sabredersiniz, iki ay sabredersiniz, üçüncü ay aboneliği iptal edersiniz. Bu yüzden gecikmeye duyarlı sistemlerin öncelikli olması gerekir.

Öncelik seviyeleri sabit değildir. Bir cihaza bir kez 2 numaralı öncelik verildi diye hep öyle kalmaz; duruma ve zamana göre değiştirilebilir. Hoca dinamik olarak değiştirilebildiği sürümler olduğunu da söyledi.

10. IEEE 802.5 Token Ring

10.1 Topoloji ve Token

Token Ring'de topoloji ağaç ya da bus değil, halkadır. Hoca halka topolojisini, bus'ın iki ucunun birleşmiş hali olarak hatırlattı: Bütün istasyonlar bir halkanın çevresine bağlıdır.

Temel yaklaşım Token Bus ile aynıdır. Token'ı sağımdan ya da solumdan alırım. Veri göndermek istiyorsam token'ı tutarım ve göndermek istediğim veriyi hatta basarım. Token Ring'de token'ın boyutu 3 bayttır. Hat boşken bile token her zaman hatta dolaşır.

10.2 Bir Veri Çerçevesinin Halkadaki Yolculuğu

Hoca token'ı hattan alıp yerine kendi verisini basmaya "token resizing" dedi. Bunun anlamı şudur: "Benden başka kimse şu an göndermesin; benim verimi alıcı adrese kadar herkes iletsin."

İşleyiş şöyledir:

1. Gönderici token'ı hattan alır ve verisini hatta basar. Çerçeve, örneğin sağdan başlayarak halkada dolaşmaya başlar.
2. Her istasyonun NIC'i çerçeveyi açar ve bakar: "Bana mı gelmiş?" Gelmemişse yanındakine iletir. Bir sonraki de açar, bakar, kendisine değilse iletir.
3. Çerçeve hedef istasyona geldiğinde hedef, çerçevenin bir kopyasını alır. Kopya üzerinden hata denetimi yapar ve HDLC'den hatırladığımız akış denetimi işlemlerini (verinin doğru gelip gelmediği, kaçıncı sıradaki veri olduğu gibi) yürütür.
4. Hedef, cevabını hâlâ elinde tuttuğu çerçevenin denetim kısmına yük bindirme (piggybacking) ile yerleştirir ve çerçeveyi yanındakine iletir. Yani veri kopyalanmıştır, ama çerçeve hâlâ halkada dolaşmaktadır.

5. Halka tamamlanınca çerçeve, onu hatta basan göndericiye döner. Gönderici, hedefin veriyi aldığını ya da hatalı aldığını yazdığını görür. Örneğin doğru alındıysa ve başka bir şey göndermeyecekse çerçeveyi hattan çeker ve yerine token'ı koyar.

   Hocanın vurgusu: Veriyi hattan, veriyi alan değil, veriyi koyan istasyon çeker.

Hoca farklı bir token alıp farklı boyutlu bir veri koymaya da "token resizing" dendiğini söyledi.

> **Ders sonrası not:** Hocanın "token resizing" dediği işlem, standartta büyük olasılıkla token yakalama (token capture) olarak geçer. Gönderici, token'ın AC baytındaki T (token) bitini değiştirerek onu bir çerçeve başlangıcına dönüştürür ve arkasından kendi alanlarını ekler. Çerçeveyi bitirince yeniden 3 baytlık bir token üretir.

10.3 Bir Bitin Fiziksel Uzunluğu ve Halkadaki Bit Sayısı

Hoca hattın performansını hesaplamak için kullanılan bir kavramı, bir bitin fiziksel uzunluğunu (physical length of a bit) anlattı. Soru şuydu: L metre uzunluğundaki halka topolojili bir hatta aynı anda kaç bit bulunabilir?

Bunun için şu büyüklükler verilmiş kabul edilir:

- **İletim hızı (transmission speed)** R**:** Hat çok hızlıysa bitler hatta "girip çıkar". Bu yüzden hızın önemi vardır.
- **Bit çıkış süresi (bit extraction):** Bir bitin, hattın hızına göre hatta konması ya da hattan çıkması için geçen süredir: $1/R$. Hattın hızıyla ters orantılıdır.
- **Sinyal yayılma hızı (signal propagation speed)** SP**:** Gönderilen sinyalin hat üzerinde ne kadar hızla ilerlediğidir.

Bir bitin halkada kapladığı uzunluk, bit süresi boyunca sinyalin aldığı yoldur:

$$\text{bit başına uzunluk} = \frac{SP}{R} \quad \text{(metre)}$$

L metrelik halkada aynı anda bulunan bit sayısı, halka uzunluğunun bit başına uzunluğa bölümüdür. Metreler birbirini götürür:

$$B = \frac{L}{SP/R} = \frac{L \cdot R}{SP}$$

Hoca bunun düşünerek de çıkarılabilecek basit bir formül olduğunu söyledi.

10.4 Öncelik ve Rezervasyon

> **Hocanın vurgusu:** Token Ring'deki öncelik ve rezervasyon mekanizmasını iyi dinleyin. Mülakatta sorulur; iş hayatında "Token Ring'deki rezervasyona benzer bir şey yap" diye mutlaka karşınıza çıkar.

Öncelik ve rezervasyon, token'ın içindeki erişim denetimi (AC, Access Control) baytıyla gerçekleştirilir. Hoca mekanizmayı bir örnekle anlattı. Halkada 30 istasyon var ve istasyonlar 0, 2, 4, 6 seviyelerinde. 30 istasyon olduğu için aynı seviyede birden fazla istasyon bulunur: Birkaç tane 6, birkaç tane 0, 2 ve 4.

Önemli nokta şudur: İstasyon, veri göndermeden önce veri göndereceğini token'ın içine yazar.

1. Önceliği 0 olan bir istasyon token'ı alıyor ve AC kısmına bakıyor. Orada 2 yazıyor. Bu, kendisinden önce önceliği 2 olan bir istasyonun veri gönderme isteğinde bulunduğu anlamına gelir. Kendisi 0 olduğu için bekler ve token'ı yanına iletir.
2. Token önceliği 4 olan, veri göndermek isteyen bir cihaza geliyor. AC kısmında 2 görüyor. Kendi seviyesi 4 olduğu için 2'yi silip 4 yazıyor; "artık ben göndereceğim" diyor ve token'ı yanına iletiyor.
3. Yandaki istasyonun seviyesi 2. Veri göndermek istese de hiçbir şey yapamaz ve token'ı iletir.
4. Sıradaki istasyonun seviyesi 6. 4'ü çizip 6 yazıyor ve token'ı iletiyor.

5. Bütün halka bu şekilde dolaşılır. Ardından en öncelikli istasyon token'ı alır.

Peki aynı seviyede birden fazla istek varsa? Ben önceliği 6 olan istasyonum ve AC kısmına 6 yazdım. Token'ı yanıma verdim. Yandaki istasyonun seviyesi de 6 ve o da veri göndermek istiyor. Ama orada zaten 6 yazıyor; bu 6 bir rezervasyon anlamına gelir. En yüksek seviyede birden fazla istasyon varsa, bu talepte ilk bulunan, yani oraya 6 değerini ilk yazan istasyon gönderme hakkını kazanır. Buna rezervasyon denir.

Hoca iki kavramı şöyle tekrarladı:

- **Öncelik (priority):** 0, 2, 4, 6 arasından en yüksek seviyeye sahip istasyonun göndermesi istenir.
- **Rezervasyon (reservation):** Herhangi bir seviyede birden fazla istek varsa hak, isteği ilk yapana tanınır.

  Ders sonrası not: Hoca Token Ring için de 0, 2, 4, 6 seviyelerini kullandı. Bu dört öncelik sınıfı Token Bus'a aittir. Token Ring'de AC baytında büyük olasılıkla 3 bitlik bir öncelik alanı ve 3 bitlik bir rezervasyon alanı vardır; bu yüzden seviyeler 0–7 arasındadır. Mekanizmanın mantığı derste anlatıldığı gibidir: Daha yüksek öncelikli istasyon rezervasyon alanına kendi seviyesini yazar.

  Ek bilgi (derste anlatılmadı): Token Ring'de AC baytı `PPPTMRRR` biçimindedir. `PPP` token'ın o anki öncelik seviyesi, `T` token bitidir (0 ise token, 1 ise veri çerçevesi). `M` izleme biti, `RRR` ise rezervasyon bitleridir. Bir istasyon token'ı ancak kendi önceliği `PPP`'den küçük değilse yakalayabilir; token'ı serbest bırakan istasyon yeni token'ın önceliğini rezervasyon alanındaki değere yükseltir.

10.5 Token Tutma Süresi ve Dinamik Öncelik

Token'ı alan istasyonun veri gönderme hakkının zaman sınırı vardır. Token belli bir zaman aralığı için istasyondadır. Hoca bunu şöyle anlattı: Token'ı aldım, süre başladı. Gönderiyorum, gönderiyorum, gönderiyorum. Süre bittiğinde token'ı bırakmam gerekir. Yani hatta yeni bir veri koymam; token'ın orijinal halini hatta yüklerim. Hoca bu zaman sınırının önemli olduğunu belirtti.

Seviyeler de dinamik olarak değişebilir. Ben önceliği 6 olan bir istasyonsam ve gönderecek çok verim varsa hep ilk sırada ben olurum. 6 olmamın bir sebebi vardır: Verimin gönderilmesinin önceliği yüksektir. 0 da düşük öncelikli olduğu için 0'dır. Ama hattın bütün kapasitesini 6'ların kullanıp 4'lere, 2'lere hiç yer kalmamasını engellemek için öncelik seviyeleri zaman zaman dinamik olarak değiştirilebilir.

10.6 İzleme İstasyonu

Halkada 30 istasyon varsa bunlardan biri izleme istasyonudur (monitoring station). İzleme istasyonu olayın akışını izler ve bir anormallik olduğunda müdahale eder.

Hoca bunun birincil-ikincil yapı gibi düşünülmemesi gerektiğini vurguladı. Birincil-ikincil yapıda birincil, ikincile mesajı doğrudan gönderir. Bir ikincil başka bir ikincile mesaj göndermek isterse mesajı birincile gönderir, birincil onu hedef ikincile iletir; cevap da yine birincil üzerinden döner. İzleme istasyonu böyle bir aracı değildir; yalnız akışı izler.

İzleme istasyonunun da başına bir şey gelebilir, örneğin ağdan kopabilir. Bu yüzden genellikle yanında yedek bir izleme istasyonu daha bulunur. İzleme istasyonunun takip ettiği iki durum vardır.

10.6.1 Token Yok (No Token Frame) Hatta her zaman bir token bulunmalıdır. Hat 30 istasyondan oluşuyorsa, her istasyonun token'ı alıp bakıp işleyip yanına vermesi belli bir süre alır. Hocanın sayısal örneği:

- **Verilen:** İstasyon başına süre 3 ms, istasyon sayısı 30.
- **Bir tur süresi:** $3 \text{ ms} \times 30 = 90$ ms. Yani token gönderildikten sonra halkayı dolaşıp geri gelmesi 90 ms sürer.
- **Güvenlik payı:** Bu süre genellikle 2 ile çarpılır: $90 \text{ ms} \times 2 = 180$ ms.
- **Sonuç:** Gönderdikten sonra 180 ms boyunca token gelmiyorsa bir sorun vardır.

Hesaplar Python ile doğrulandı. Hoca bu parametrelerin değişebileceğini, sayıları süreyi canlandırmak için verdiğini söyledi.

Token'a ne olmuş olabilir? Ağdaki bir cihaz token'ı aldı; belki işledi, belki kendisi veri gönderecekti. Herhangi bir sebeple token'ı hatta geri basmadan önce hattan koptu. Yanındakine kimse token gönderemez; token onda kaldı ve o da ağ bağlantısını kaybetti. Bu durumda hatta hiç token çerçevesi yoktur.

Birinin token göndermesi gerekir. İzleme istasyonu dışındaki cihazlar bu durumdan "şikâyetçi değildir". İzleme istasyonu ise hatta mutlaka bir token olması gerektiğini bilir. Hatta token olmadığını görürse token'ı kendisi yeniden hatta basar. İzleme istasyonunun birinci görevi budur.

10.6.2 Sahipsiz Çerçeve (Orphan Frame) İkinci görev sahipsiz çerçevelerdir (orphan frame). Hocanın örneği: Ben 3 numaralı istasyonum. Önceliğim ve rezervasyonum uygun, token bana geldi, veriyi hatta bastım; 17 numaralı istasyona göndereceğim. Çerçeve yola çıktı, ama 17 numaralı istasyonun bağlantısı ben göndermeye karar verdikten sonra koptu.

Normalde 17 numaralı istasyon çerçeveyi alıp denetim kısmına bir şeyler işlemelidir. Bu olmadığı için çerçeve halkayı dolaşır, bana döner, işlenmediği için yine dolaşır. İzleme istasyonu bunu fark eder: Çerçeve ikinci, üçüncü kez dolaşmıştır (belli bir zaman aşımı değeri vardır) ve sahibi yoktur, yani "ben alacağım" diye kimse yeltenmez. İzleme istasyonu bunun hiç kimseye ulaşamayacak bir sahipsiz çerçeve olduğuna karar verir, çerçeveyi hattan temizler ve yeniden hatta token basar.

10.7 Kodlama, Hız ve Çerçeve Yapısı

Token'lar her istasyonun ağ arayüz kartına uğrar ve orada denetlenir. Token Ring'in diğer özellikleri:

- **Kodlama:** Diferansiyel Manchester kodlama (differential Manchester coding) kullanılır.
- **Hız:** İlk sürümlerde 16 Mbps'e kadar çıkar. Hoca 802.5'in ileri sürümlerinde gigabit hızlara ulaşıldığını söyledi.
- **Bit sırası:** 802.3 ve 802.4 en az anlamlı bit (LSB, least significant bit) tabanlı çalışır; önce gönderilen bit LSB'dir. Token Ring'de ise önce en anlamlı bit (MSB, most significant bit) gönderilir.
- **Veri alanı:** 802.4'te yaklaşık 8000 bayt olan veri alanı, Token Ring'de yaklaşık 4500 bayttır.
- **Token yapısı:** Başlangıç sınırlayıcı (SD, Start Delimiter), erişim denetimi (AC) baytı ve bitiş sınırlayıcı (ED, End Delimiter) olmak üzere 3 bayttır.
- **İptal dizisi:** Hoca, örneğin sahipsiz çerçeve gibi durumlarda işlemleri iptal etmek için kullanılan, yalnız başlangıç ve bitiş sınırlayıcılarından oluşan 2 baytlık bir çerçeveden söz etti ve bunu izleme istasyonunun kullandığını söyledi.

11. FDDI (Fiber Distributed Data Interface)

11.1 Genel Özellikler

802.3 ve 802.4 gibi kablolu yapılardan sonra hoca günümüzde yaygınlaşan fiber optik ağlara geçti. Fiber optik kablo fiziksel katmanda anlatılmıştı. Bunun veri bağlantı katmanındaki karşılığı olarak FDDI (Fiber Distributed Data Interface) adlı bir yöntem vardır. Hoca bunu bir ANSI–ITU standardı olarak andı.

Fiber optik kablo kullandığı için ilk ortaya atıldığında 100 Mbps hızlara rahatça ulaşabiliyordu; bugün çok daha yüksek hızlara ulaşılabiliyor. FDDI token tabanlı çalışır: 802.5'te anlatılan token sistematiğine çok benzer.

> **Ders sonrası not:** Hoca FDDI'ı "ANSI–ITU standardı" diye andı. FDDI büyük olasılıkla ANSI X3T9.5 komitesinde geliştirildi ve sonradan ISO 9314 olarak standartlaştırıldı; ITU standardı değildir.

11.2 Senkron ve Asenkron Çerçeveler

FDDI'da iki çerçeve çeşidi vardır:

- **S-frame (synchronous frame, senkron çerçeve):** Bir öncelik mekanizmasıdır. Göndereceğiniz verinin karşıya zamanında ulaşması önemliyse bu öncelikli veridir ve S-frame'lerle gönderilir.
- **A-frame (asynchronous frame, asenkron çerçeve):** Önceliksiz veriler içindir.

Fiber tarafa geçmenin asıl amacı daha yüksek hızlara ulaşmaktır. Token tabanlı bir teknikte şu sorular önemlidir: Ne kadar süre veri gönderebilirim? Hangi zaman aralığında gönderebilirim? Bununla ilgili hesaplama teknikleri vardır; sonuçlar çeşitli yazmaçlara (register) konur ve işlem buna göre yapılır.

11.3 Gönderim Süresinin Hesabı

Hoca hesabı şöyle özetledi. Hatta yine 30 istasyon olsun. Her biri token'ı alıp açıp bakıyor, bir şeyler yapıyor. Token'ı gönderdiğim andan yeniden bana ulaştığı ana kadar geçen bir süre vardır. Bir de bana varsayılan olarak verilen, hattaki veri iletiminin ortalama süresi vardır. Bu ortalama süreden, gönderdiğim anla aldığım an arasında geçen süreyi çıkarırsam geriye "bu hatta ne kadar süre veri basabileceğim" kalır.

Örnek: Hesap sonucunda hat 17 milisaniye boyunca benim için ayrılmış olsun. Token'ı aldım, veri basacağım:

1. Önce elimdeki bütün S-frame'leri basarım.
2. Hocanın anlatımına göre bana ayrılan süre içinde S-frame'lerin hepsini gönderemediysem token'ı yeniden almak için "savaşa girerim".
3. S-frame'lerin hepsini gönderdiysem, kalan süre içinde A-frame'leri göndermeye başlarım.

Bana ayrılan sürenin tamamını kullanmak zorunda değilim. Hocanın örneğinde süre ayrılmış, ama göndereceğim veri 3 milisaniyede bitmiş; bu durumda token'ı serbest bırakabilirim.

> **Hocanın vurgusu:** FDDI'da öncelik istasyonlara değil, çerçevelere verilir. Sistem "o istasyon hep öncelikli, bunun verileri önceliksiz" demez. Aynı istasyonun bazı verileri öncelikli, bazıları önceliksizdir. Hocanın örneği: Derste gerçek zamanlı aldığınız görüntüsü ve sesi önceliklidir (S-frame). Size e-posta göndermesi ya da bir sürücüye (drive) dosya yüklemesi daha az önceliklidir (A-frame). Hepsi aynı istasyondan çıkar.
>
> **Ders sonrası not:** Hoca, senkron çerçeveler ayrılan süreye sığmazsa token için yeniden "savaşa girildiğini" söyledi. FDDI'da büyük olasılıkla her istasyonun senkron trafik için önceden ayrılmış ve garanti edilen bir süresi vardır; token'ı her aldığında bu süre kadar senkron veri gönderebilir. Hocanın tarif ettiği "beklenen dönüş süresinden gerçek dönüş süresini çıkarma" hesabı, büyük olasılıkla yalnız asenkron trafik için kullanılabilecek süreyi verir: Hedef token dönüş süresi (TTRT) ile ölçülen token dönüş süresi (TRT) arasındaki fark.

11.4 4B/5B Kodlama

> **Hocanın vurgusu:** Kodlama kısmı önemli, "can kulağıyla" dinleyin.

FDDI'da daha önce gördüğümüz NRZI (Non-Return to Zero Inverted) yöntemi kullanılır. NRZI'de art arda belli sayıda 0 gelmesi senkronizasyon açısından sorun oluşturuyordu. Hoca, ikiden fazla ardışık 0'ın problemli olduğunu söyledi.

Çözüm olarak her 4 bitlik veri 5 bitlik bir kodla ifade edilir. Bu 5 bitlik kodlar, hocanın ifadesiyle, ikiden fazla, yani 3, 4, 5 tane 0 art arda gelmeyecek şekilde seçilir.

Sayım şöyledir: 4 bitle $2^4 = 16$ farklı durum, 5 bitle $2^5 = 32$ farklı durum ifade edilebilir. Veri için 16 kod yeterlidir. Kalan 16 kodun bir kısmı denetim (control) sembolleri için kullanılır. Denetim kodları belki ardışık ve tekrarlı gönderileceği için bunlarda ikiden fazla 0'ın yan yana geldiği durumlar görülebilir.

Bedeli şudur: Veri kısmında her 4 bit için karşıya 5 bit gönderilir. 16 bit göndereceğim yere 20 bit gönderirim: $16/4 \times 5 = 20$ (ara adım eklendi). Bu, hattın veri kısmındaki verimde yöntemden kaynaklanan bir dezavantajdır; ek yük $5/4 - 1 = \%25$'tir (ara adım eklendi). Ama fiber zaten hızlı çalıştığı için, uygun bir modülasyonla daha yüksek performans elde etme amacıyla ortaya atılmıştır.

Hocanın vurgusu: Her 4 bit yerine 5 bit gönderilir. 16 bit veri için 20 bit gönderilir.

Kodlama tablosu şöyledir:

Veri	Kod	Veri	Kod
0000	11110	1000	10010
0001	01001	1001	10011
0010	10100	1010	10110
0011	10101	1011	10111
0100	01010	1100	11010
0101	01011	1101	11011
0110	01110	1110	11100
0111	01111	1111	11101

Hocanın örnekleri:

- **Gönderici:** Göndereceği ilk 4 bit `0100` ise onun yerine tablodaki 5 bitlik kodu gönderir.
- **Alıcı:** Karşısına `11011` çıktıysa tabloya bakar ve göndericinin aslında `1101` göndermek istediğini anlar (Python ile doğrulandı).

  Ders sonrası not: Hoca `0100` için `0101` gönderildiğini söyledi; bu 4 bitlik bir değerdir. Tabloya göre `0100` verisinin kodu `01010`'dur. Hocanın "ikiden fazla ardışık 0 gelmez" kuralı da kod akışı için tam doğru değildir. Veri kodlarında baştaki 0 sayısı en çok 1, sondaki 0 sayısı en çok 2'dir. Bu yüzden iki kod art arda geldiğinde en fazla 3 ardışık 0 oluşur. Bu, 16 kodun bütün ikili birleşimleri denenerek Python ile doğrulandı.

Çerçeve yapısı Token Ring'e çok benzer; FDDI token'ı da Token Ring yöntemindeki token'a çok benzer.

11.5 Çift Bağlı ve Tek Bağlı İstasyonlar

FDDI'da çift modda (dual) ve tek modda (single) çalışan istasyonlar vardır.

Çift bağlı istasyon (DAS, Dual Attachment Station): İki ortam arayüz denetleyicisi (kartı) ve iki hattı vardır. Hatlardan biri bir yönde, diğeri ters yönde döner. Hoca hatlardan birinin belki senkronizasyon, diğerinin veri gönderimi için kullanıldığını, iki hattın da erişim için kullanıldığını ve bu sayede performansın daha yüksek olduğunu söyledi.

Ama hemen akla bir sorun gelir: maliyet. Her istasyonda iki ağ kartı gerekir, üstelik fiber optik için; bu para demektir. İki hat çekilmesi gerekir; bu da para demektir. Bu yüzden bütün istasyonlar böyle değildir.

Tek bağlı istasyon (SAS, Single Attachment Station): Tek bir ortam arayüz denetleyicisi vardır. Bunların çift halkalı yapıya bağlanabilmesi için arada köprü görevi gören birimler vardır. Hoca bunları DAC olarak andı: Bir tarafı çift çalışmaya, diğer tarafı tek çalışmaya uygun bağlantılardan oluşur; SAS'lar buna bağlanarak halka döngüsüne katılır. Bir de SAC birimleri vardır: Tek bağlıları birbirine bağlar; ana (master) gibi çalışır, ona n tane bağımlı (slave) birim bağlanır ve bu n birim bir arada tek bir SAS gibi davranarak bir DAC'a bağlanır ve çift bağlılarla iletişim kurar.

Özetle mimaride DAS, DAC, SAS ve SAC birimleri vardır. D'ler "dual" kelimesinden gelir.

> **Ders sonrası not:** Hoca DAC ve SAC'deki C harfinin açılımını hatırlamadığını, "converter" (dönüştürücü) olabileceğini söyledi. C, yoğunlaştırıcı (concentrator) anlamına gelir: DAC çift bağlı yoğunlaştırıcı, SAC tek bağlı yoğunlaştırıcıdır. Hoca ayrıca iki halkadan birinin senkronizasyon, diğerinin veri için kullanıldığını söyledi; büyük

olasılıkla normal çalışmada veri birincil halkada akar, ikincil halka yedektir ve bir kopukluk olunca halka ikincil halka üzerinden kapatılır.

Örnek Sorular

Soru 1 (derste sözlü, 00:18:37): Slotlu ALOHA'da herkes kendi zamanında gönderiyorsa çarpışma olmaması ve kullanım oranının %100 olması gerekmez mi?

Derste verilen cevap: Hoca saatlerin elektromanyetik etkiler gibi nedenlerle kaydığını, bir istasyonun başkasının slotuna taştığını ve bu yüzden çarpışma olduğunu söyledi; kullanım oranı %37'de kalır.

> **Çözüm (derste verilmedi):** Slotlu ALOHA'da slotlar istasyonlara sabit olarak dağıtılmaz; her istasyon herhangi bir slotun başında gönderebilir. Aynı slotu seçen iki istasyon çarpışır. Sunulan yük G iken başarılı iletim oranı $S = Ge^{-G}$'dir. En büyük değer $G = 1$'de $S = 1/e \approx 0{,}368$ olur (Python ile doğrulandı). Yani %37 sınırı saat kaymasından değil, rastgele erişimin kendisinden gelir.

Soru 2 (derste sözlü, 00:24:54): Gönderim sırasında oluşan çarpışmayı algılayabilir miyim? Çarpışma olduğunda ne yapılır?

Derste verilen cevap: CSMA/CD'de çarpışma algılanır, bir jam sinyali gönderilir ve istasyon ikili üstel geri çekilme algoritmasının ürettiği süre kadar bekleyip işlemi yeniden dener.

Soru 3 (derste sözlü, 01:08:20): L metre uzunluğundaki halka topolojili bir hatta aynı anda kaç bit bulunabilir?

Derste verilen cevap: Bir bit hatta SP/R metre yer kaplar. Halkadaki bit sayısı $B = \dfrac{L \cdot R}{SP}$'dir.

Derste sayısal değer verilmedi.

Soru 4 (derste sözlü, 01:16:21): Halkada 30 istasyon var ve her istasyonda token yaklaşık 3 ms kalıyor. Token gönderildikten sonra 180 ms geçtiği halde geri gelmiyorsa token'a ne olmuştur?

Derste verilen cevap: Bir tur $30 \times 3 = 90$ ms sürer; güvenlik payıyla $2 \times 90 = 180$ ms beklenir (Python ile doğrulandı). Token'ı alan bir istasyon, token'ı hatta geri basmadan ağdan kopmuş olabilir. Hatta token kalmamıştır. Bu durumu izleme istasyonu fark eder ve hatta yeni bir token basar.

Dersin Toparlaması

Bu derste veri bağlantı katmanının yerel ağlardaki uzantısı incelendi. IEEE 802 projesi, farklı ağ kartlarının ve protokollerin uyumlu çalışabilmesi için veri bağlantı katmanını LLC ve MAC alt katmanlarına ayırır. Paylaşılan bir ortamda asıl sorun çarpışmadır. ALOHA'dan slotlu ALOHA'ya, oradan hattı dinleyen CSMA türlerine ve çarpışmayı algılayıp geri çekilen CSMA/CD'ye uzanan zincir, bu sorunu azaltma çabasının adımlarıdır. Ethernet (802.3) bu zincirin sonunda çıkan, çerçeve yapısı, sürümleri ve PoE gibi uzantılarıyla bugün en yaygın LAN teknolojisidir.

Dersin ikinci yarısında çarpışmayı tamamen ortadan kaldıran token tabanlı yöntemler anlatıldı. Token Bus (802.4) fiziksel bir bus üzerinde mantıksal bir sıra kurar; Token Ring (802.5) halka üzerinde token'ı dolaştırır, öncelik ve rezervasyonu AC baytıyla yönetir ve izleme istasyonuyla kaybolan token'ı ve sahipsiz çerçeveleri düzeltir. FDDI aynı token mantığını fiber üzerine taşır, önceliği istasyona değil çerçeveye verir ve NRZI'nin senkronizasyon sorununu 4B/5B kodlamayla çözer. Bir sonraki derste kablosuz yerel ağlar (802.11) anlatılacak.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.